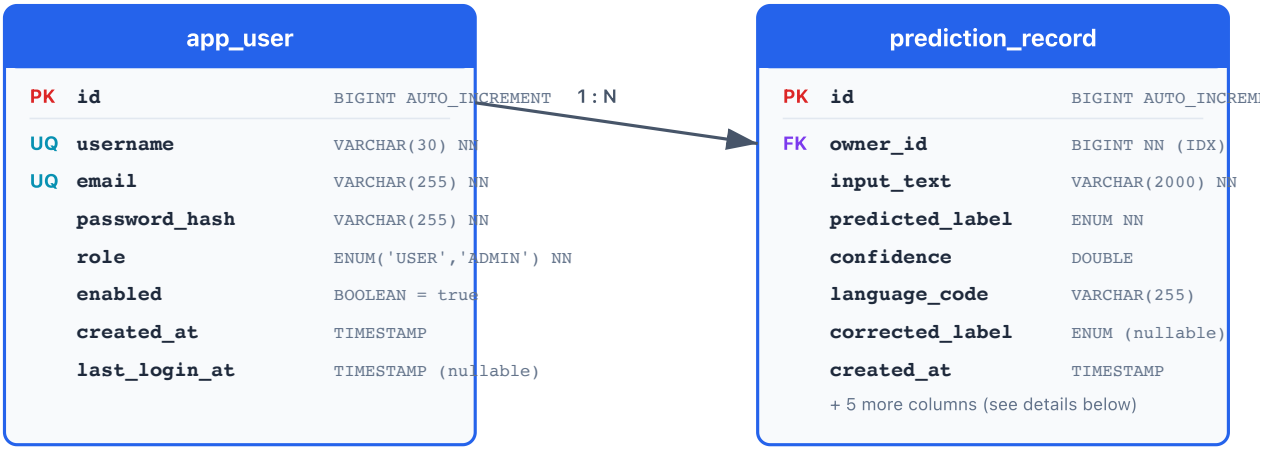
Database Architecture

Sentiment Analysis Application — MySQL 8.x • JPA / Hibernate • Spring Boot 3.5

Overview

- **Database:** MySQL 8.x (database name: `sentimentdb`)
- **Schema management:** Hibernate `ddl-auto: update` — tables are created automatically on first run if they do not exist
- **Tables:** 2 — `app_user`, `prediction_record`
- **Relationship:** One-to-Many — one user owns many prediction records
- **Test environment:** H2 in-memory database (via `application-test.yml` with `spring.profiles.active=test`)

Entity Relationship Diagram



`app_user` Entity: AppUser.java

Stores registered users (both regular users and administrators). Created at registration; updated on login.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	BIGINT	PK NOT NULL	Auto-incremented primary key (GenerationType.IDENTITY)
username	VARCHAR(30)	UNIQUE NOT NULL	Login name, max 30 characters
email	VARCHAR(255)	UNIQUE NOT NULL	User's email address
password_hash	VARCHAR(255)	NOT NULL	BCrypt-hashed password
role	VARCHAR(255)	NOT NULL DEFAULT 'USER'	Authorization role, stored as string via `@Enumerated(STRING)`. Values: `USER`, `ADMIN`
enabled	BOOLEAN	DEFAULT true	Account active flag; can be toggled by admin
created_at	TIMESTAMP		Registration timestamp (set to `Instant.now()` on creation)
last_login_at	TIMESTAMP	NULLABLE	Last successful login; null until first login

prediction_record

Entity: PredictionRecord.java

Stores every sentiment analysis run. Each record belongs to exactly one user via the `owner_id` foreign key. An index on `owner_id` (`idx_prediction_owner`) accelerates per-user history queries.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	BIGINT	PK NOT NULL	Auto-incremented primary key
owner_id	BIGINT	FK NOT NULL	References `app_user.id` — the

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
		INDEX	user who ran this analysis
input_text	VARCHAR(2000)	NOT NULL	The original text submitted for analysis
language_code	VARCHAR(255)	NULLABLE	Detected language code (e.g., `ta`, `te`, `kn`, `ml`, `und-Latn`)
selected_language	VARCHAR(255)	NULLABLE	Language the user picked on the Analyze page; null when left on auto-detect
script	VARCHAR(255)	NULLABLE	Detected script name (e.g., Tamil, Latin)
code_mixed	BOOLEAN		Whether the input contained code-mixed text (multiple scripts)
predicted_label	VARCHAR(255)	NOT NULL	Model's prediction, stored as string via `@Enumerated(STRING)`. Values: `POSITIVE`, `NEGATIVE`, `NEUTRAL`
confidence	DOUBLE		Model confidence score (0.0 – 1.0)
low_confidence	BOOLEAN		Flagged when confidence is below the configured threshold (default 0.60)
latency_ms	BIGINT		Inference time in milliseconds
corrected_label	VARCHAR(255)	NULLABLE	Human-corrected label (set when a user or admin overrides the

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
			prediction). Values: `POSITIVE`, `NEGATIVE`, `NEUTRAL`
created_at	`TIMESTAMP`		When the analysis was performed (set to `Instant.now()` on creation)

Enumerated Types (stored as VARCHAR via @Enumerated(STRING))

Role

`USER` `ADMIN`

Sentiment

`POSITIVE` `NEGATIVE` `NEUTRAL`

application.yml — Database Configuration

```
spring.datasource.url:
jdbc:mysql://${DB_HOST:localhost}:${DB_PORT:3306}/${DB_NAME:sentimentdb}?
createDatabaseIfNotExist=true
spring.datasource.username: ${DB_USERNAME:root}
spring.datasource.password: ${DB_PASSWORD:root}
spring.datasource.driver-class-name: com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto: update  # creates tables if missing, never
drops
spring.jpa.properties.hibernate.dialect: org.hibernate.dialect.MySQLDialect
```

Key Design Notes

- **Auto DDL:** `ddl-auto: update` creates tables on first run if they don't exist; it never drops or recreates existing tables
- **Auto DB creation:** `createDatabaseIfNotExist=true` in the JDBC URL creates the `sentimentdb` schema if it's missing
- **Naming strategy:** Hibernate's default `SpringPhysicalNamingStrategy` converts camelCase fields to snake_case columns (e.g., `passwordHash` → `password_hash`)

- **Lazy loading:** The `owner` relationship on `prediction_record` uses `FetchType.LAZY` to avoid N+1 queries
- **Index:** `idx_prediction_owner` on `owner_id` optimizes user-specific history queries and dashboard aggregations
- **Test isolation:** Tests use the `test` profile which switches to H2 in-memory database, avoiding the MySQL dependency in CI/CD